Plan Técnico: Modo de Renderizado Dual 3D en Vivo (WebGL / Three.js)

1. Visión General y Objetivo de Ingeniería

Hasta ahora, la plataforma unificada permite alternar la visualización en vivo entre la Cámara 1 y la Cámara 2 mediante un selector en la interfaz, y realizar capturas duales estáticas donde Open3D fusiona ambas nubes de puntos.

El **Modo de Renderizado Dual en Vivo** lleva esta capacidad al siguiente nivel:

> **Objetivo:** Transmitir y renderizar simultáneamente en tiempo real (30 FPS) la geometría 3D generada por **ambas cámaras RealSense en el mismo visor WebGL interactivo (Three.js)**, visualizando la escena completa con cobertura expandida (o 360°) de manera continua y fluida.

---

2. Modos de Visualización en el Visor 3D

Para brindar máxima flexibilidad diagnóstica y de inspección, el visor WebGL soportará 4 modos de renderizado seleccionables en caliente:

A. Modo Co-Renderizado Fusionado 3D (Dual Fusion View - Modo Principal)

- **Concepto:** Ambas cámaras proyectan sus nubes de puntos/mallas en el **mismo espacio tridimensional global**.
- **Posicionamiento:**
- La **Cámara 1 (Master)** se ubica en el origen de coordenadas mundial $(0,0,0)$.
- La **Cámara 2 (Slave)** se posiciona y orienta en el espacio 3D aplicando en tiempo real la matriz extrínseca de calibración $T_{2 \to 1} \in SE(3)$.
- **Colorimetría:**
- **RGB Fotorrealista:** Cada cámara proyecta sus propios colores de textura reales sobre su geometría.
- **Diagnóstico de Solape (Color por Cámara):** La Cámara 1 se colorea en tono cian translúcido y la Cámara 2 en tono ámbar/magenta. Permite al usuario verificar instantáneamente si la calibración espacial es perfecta inspeccionando cómo se cruzan y encajan las superficies comunes.
- **Normales de Superficie / Clay:** Iluminación uniforme para evaluar volumen y relieve sin distracción de textura.

```
        [ Visor WebGL Único · Escena 3D Global ]
                              |
         +-----------------+-----------------+
         |                                   |
  Nodo Cámara 1                        Nodo Cámara 2
  - Posición: [0, 0, 0]                - Posición: T(2 -> 1)
  - Rotación: Identidad                - Rotación: R(2 -> 1)
  - Texturas: Prof1 + Col1             - Texturas: Prof2 + Col2
  - Shaders GPU (Puntos/Malla)         - Shaders GPU (Puntos/Malla)
```

B. Modo Pantalla Dividida (Split-Screen Viewport)

- El canvas WebGL se divide verticalmente en dos cuadrantes mediante las APIs de Three.js `renderer.setViewport` y `renderer.setScissor`:
- **Lado Izquierdo (50% W):** Vista 3D de la Cámara 1.
- **Lado Derecho (50% W):** Vista 3D de la Cámara 2.
- Permite comparar el campo de visión de cada cámara individualmente con controles de cámara orbital sincronizados o independientes.

C. Modo Imagen sobre Imagen (Picture-in-Picture / PiP)

- El visor muestra la vista 3D completa de la cámara principal, mientras que un mini-viewport flotante en una esquina muestra la perspectiva de la segunda cámara en tiempo real.

D. Modo Cámara Única (Solo Cam 1 / Solo Cam 2)

- Modo estándar de bajo consumo que desactiva el renderizado secundario para ahorrar recursos cuando sólo se requiere calibrar un ángulo específico.

---

3. Arquitectura del Protocolo de Red y Streaming Multi-Cámara

A. Multiplexación en el WebSocket Único (`/ws`)

Para evitar la sobrecarga de gestionar múltiples conexiones WebSocket y reconexiones desincronizadas, ambos streams de vídeo 3D se envían a través del **mismo socket** `/ws` mediante multiplexación de paquetes binarios.

Estructura de la Cabecera Binaria (32 Bytes):

El protocolo `RSF1` definido en `live3d/protocol.py` cuenta con campos clave para identificar la fuente:

```
Cabecera RSF1 (32 bytes Little-Endian):
 Byte 0..3:   Magic (b"RSF1")
```

```
Byte 4..7:   Número de secuencia (u32 seq)
Byte 8..15:  Timestamp del servidor (f64 segundos)
Byte 16..17: ID de Metadatos (u16 meta_id: 1 para Cam 1, 2 para Cam 2)
Byte 18:     Codec de profundidad (0=raw, 1=zlib)
Byte 19:     Codec de color (0=none, 1=jpeg, 2=raw)
Byte 20..23: Longitud del bloque de profundidad (u32 depth_len)
Byte 24..27: Longitud del bloque de color (u32 color_len)
Byte 28..31: Flags / Stream ID (u32 flags: bit 0 = cam_idx: 0 ó 1)
```

- **Flujo de Envío:**
- `CamWorker-Master` captura frame $\to$ Codifica paquete etiquetado con `cam_idx = 0` y `meta_id = 1` $\to$ Envía por WebSocket.
- `CamWorker-Slave` captura frame $\to$ Codifica paquete etiquetado con `cam_idx = 1` y `meta_id = 2` $\to$ Envía por WebSocket.
- **Flujo de Recepción en el Cliente JS (`stream.js` y `viewer.js`):**
- El despachador lee el campo `cam_idx` (vía `meta_id` o `flags`).
- Si `cam_idx == 0`, actualiza las texturas de la GPU (`uDepth1`, `uColor1`) del nodo de la Cámara 1.
- Si `cam_idx == 1`, actualiza las texturas de la GPU (`uDepth2`, `uColor2`) del nodo de la Cámara 2.

B. Presupuesto de Ancho de Banda y Rendimiento

Flujo	Resolución	Compresión	Tasa de Datos por Cámara	Tasa Dual (2 Cámaras)
Profundidad	848x480 @ 30 FPS	zlib nivel 1	~1.5 MB/s (12 Mbps)	~3.0 MB/s (24 Mbps)
Color	848x480 @ 30 FPS	JPEG Q75	~1.8 MB/s (14.4 Mbps)	~3.6 MB/s (28.8 Mbps)
Total Red	-	-	**~3.3 MB/s (26.4 Mbps)**	**~6.6 MB/s (52.8 Mbps)**

> [!NOTE] Un consumo total de **6.6 MB/s** es completamente insignificante para el loopback local (`localhost`), donde el ancho de banda del stack TCP de Windows supera los 1.500 MB/s. El consumo de CPU en el servidor se mantiene por debajo del 12% gracias a la compresión multihilo.

---

4. Transformación de Coordenadas: RealSense a Three.js

Las cámaras RealSense y el motor Three.js utilizan convenciones de sistemas de coordenadas diferentes: * **Sistema RealSense (Métrico):** - $+X$: Hacia la derecha de la cámara. - $+Y$: Hacia abajo (coincide con filas de imagen). - $+Z$: Hacia adelante (profundidad perpendicular al sensor). * **Sistema Three.js (Métrico):** - $+X$: Hacia la derecha. - $+Y$: Hacia arriba. - $+Z$: Hacia el observador (atrás).

Para proyectar la Cámara 2 en Three.js con la matriz extrínseca de calibración $T_{2 \to 1}^{\text{RS}}$ obtenida por OpenCV solvePnP:

1. Se define la matriz de cambio de base C: $$C = \begin{bmatrix} 1 & 0 & 0 & 0 \ 0 & -1 & 0 & 0 \ 0 & 0 & -1 & 0 \ 0 & 0 & 0 & 1 \end{bmatrix}$$
2. La transformación en el espacio de Three.js es: $$T_{\text{ThreeJS}} = C \cdot T_{2 \to 1}^{\text{RS}} \cdot C^{-1}$$
3. En el grafo de escena de Three.js: `javascript // Desactivar cálculo automático para inyectar la matriz 4x4 directamente cam2Group.matrixAutoUpdate = false; cam2Group.matrix.copy(T_threejs); cam2Group.matrixWorldNeedsUpdate = true;` De este modo, cualquier vértice generado por la Cámara 2 se transforma automáticamente en la GPU al sistema de referencia de la Cámara 1.

---

5. Arquitectura del Visor WebGL (`viewer.js` Refactorizado)

Para soportar el renderizado dual eficiente sin duplicar código, se introduce la clase `CameraRenderNode`:

```
class CameraRenderNode {
 constructor(scene, camIdx, isMaster) {
   this.camIdx = camIdx;
   this.group = new THREE.Group();
   scene.add(this.group);

   // Geometría compartida (Rejilla W x H vértices)
   this.geometry = null;
   this.depthTexture = null;
   this.colorTexture = null;

   // Shader de puntos y malla con uniforms dedicados
   this.uniforms = { ...commonUniforms, uCamIdx: camIdx };
   this.pointsMesh = new THREE.Points(this.geometry, this.pointsMaterial);
   this.mesh = new THREE.Mesh(this.geometry, this.meshMaterial);
   this.group.add(this.pointsMesh, this.mesh);
 }

 setMeta(meta) { ... }
 updateTextures(depthBuffer, colorBitmap) { ... }
 setTransform(matrix4x4) {
   this.group.matrixAutoUpdate = false;
   this.group.matrix.fromArray(matrix4x4);
   this.group.matrixWorldNeedsUpdate = true;
 }
}
```

El objeto `Viewer` principal mantiene: * `this.cam1Node = new CameraRenderNode(scene, 0, true);` * `this.cam2Node = new CameraRenderNode(scene, 1, false);` * En cada fotograma recibido por WebSocket, se delega al nodo correspondiente.

---

6. Panel de Ajuste Fino Extrínseco Interactivo (Gizmo & Sliders)

Aunque la calibración ChArUco calcula $T_{2 \to 1}$ de forma automática, las tolerancias ópticas o flexiones mecánicas del soporte pueden requerir ajustes de décimas de milímetro. El visor incorporará: 1. **Controles deslizantes (Sliders) en tiempo real:** - Traslación: X, Y, Z (rango $\pm 10\ \text{cm}$ con precisión de $0.5\ \text{mm}$). - Rotación: Yaw, Pitch, Roll (rango $\pm 15^\circ$ con precisión de 0.1°). 2. **Efecto Inmediato:** Al mover cualquier control, la matriz T_{ThreeJS} de la Cámara 2 se actualiza en el siguiente cuadro de la GPU, permitiendo ver el alineamiento instantáneamente. 3. **Botón "Guardar Calibración":** Envía la matriz ajustada al servidor vía `POST /api/dual/extrinsics` para persistirla en `config/dual_camera_config.json`.

---

7. Plan de Implementación por Fases

Fase	Tarea Principal	Archivos Afectados	Entregable
Fase 1	**Multiplexación de Red**	`live3d/protocol.py`, `live3d/capture.py`, `live3d/ser`	Servidor emitiendo frames de ambas cámaras etiquetados por WebSocket.
Fase 2	**Nodos de Renderizado Three.js**	`live3d/web/js/viewer.js`, `live3d/web/js/shaders.js`	Soporte para dos mallas/nubes concurrentes con uniforms independientes.
Fase 3	**Despacho en Cliente Web**	`live3d/web/js/stream.js`, `live3d/web/js/main.js`	Enrutamiento de paquetes binarios al nodo correspondiente sin caída de FPS.
Fase 4	**Interfaz de Usuario y Modos**	`live3d/web/index.html`, `live3d/web/css/style.css`	Selector "Dual Fusion / Split / Solo", colores de solape y sliders de ajuste.

Fase	Tarea Principal	Archivos Afectados	Entregable
Fase 5	**Validación y Pruebas**	Suite de pruebas y navegador	30 FPS en ambas cámaras, 60 FPS en WebGL y exportación de documentación en PDF.